

F7LY内核介绍

2026年全国大学生计算机系统能力大赛 操作系统设计赛(全国) OS内核实现赛道

小队成员：曹子宸、郑喆宇、何佳怡

指导老师：蔡朝晖

整体架构

Overall Architecture



1.1 参考项目与复用边界

◆ 2025武汉大学队伍F7LY

- 项目基线，本队去年参赛作品
- 开发仓库相同，本年度开发从2026年1月开始

◆ LWEXT4

- 移植c语言ext4库

◆ Open-NPStack

- 移植开源网络协议栈

◆ 内存分配器

- 采用LibAllocator和Buddy算法

◆ 2023华中科技大学队伍AVX

- 移植了sdcard驱动

◆ EASTL

- 作为第三方库

◆ Starry-OS:

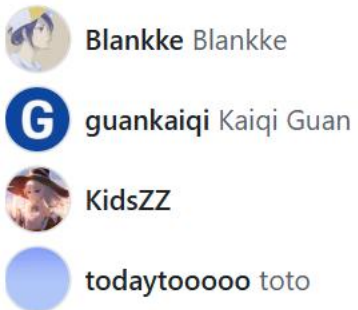
- 在 I/O 调度和性能优化思路提供了横向对比参考



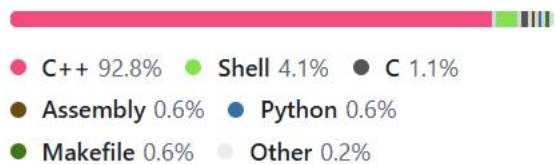
1.2 项目概览



Contributors 4

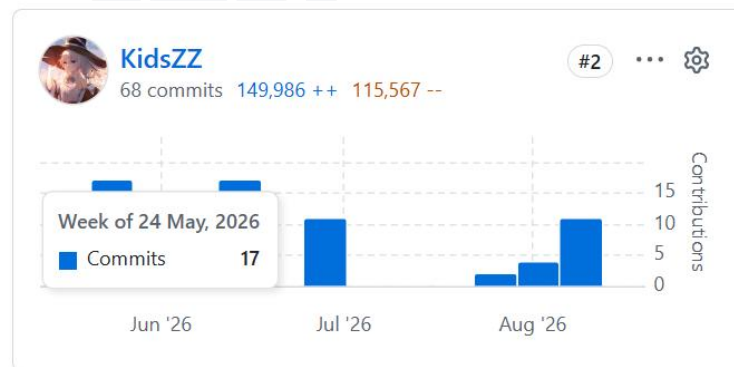


Languages



GitHub 完整保留核心代码、平台适配与兼容性修复的开发记录，形成可追溯的工程演进过程。

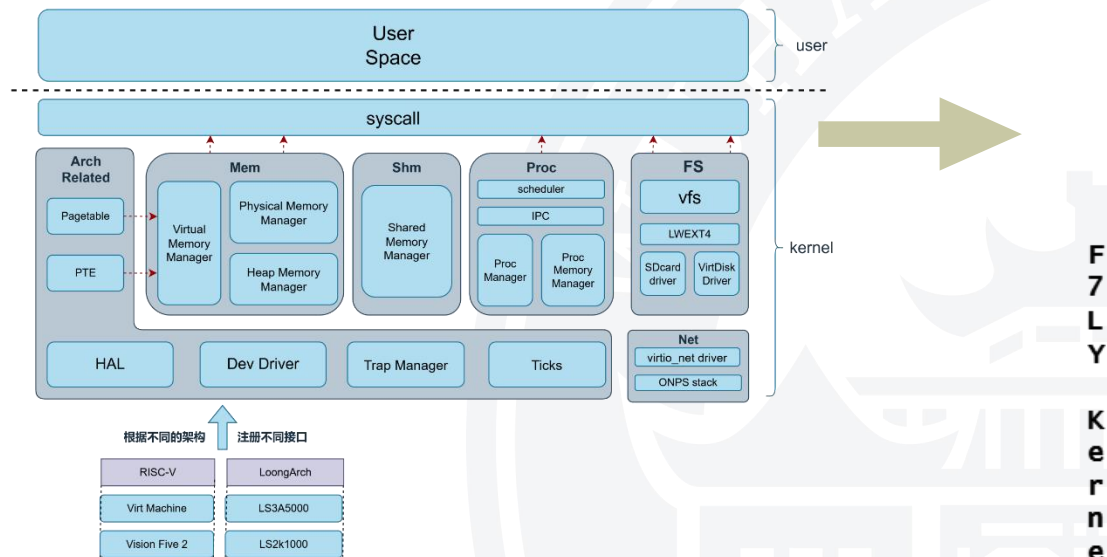
项目地址: github.com/Blankke/F7LY



后续介绍中我们在基线的基础上的工作均为增量，覆盖多核、网络、linux语义等性能提升部分

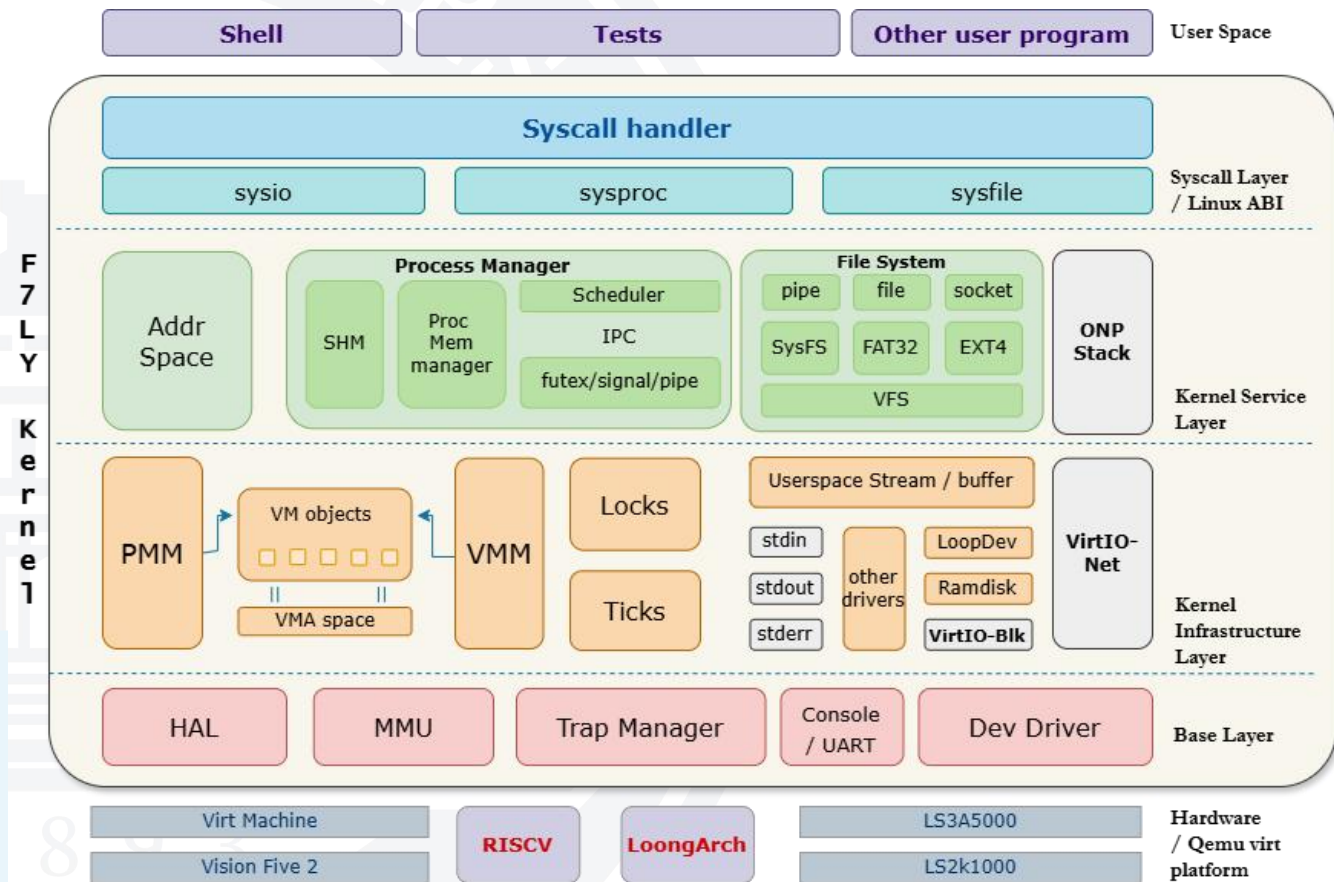


1.3 分层内核架构



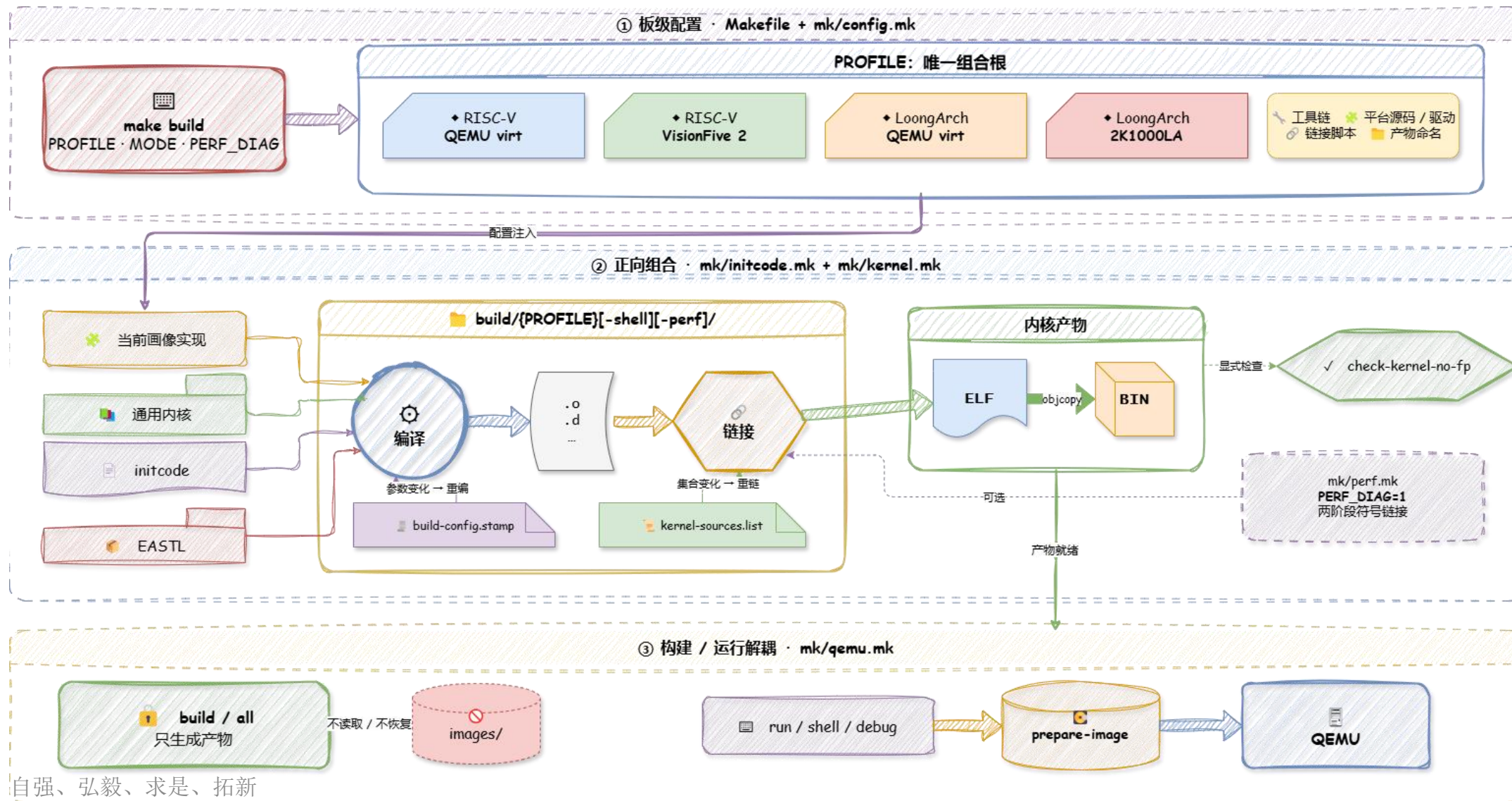
- 服务层按地址空间、进程/IPC、文件系统与网络模块组织，系统调用层统一暴露 Linux ABI
- RISC-V / LoongArch 的差异收敛到 HAL、MMU、Trap 与平台层，上层核心机制共享实现
- 补齐 VMA / VMObject、VFS、多核调度与网络链路，扩大可运行 Linux 用户程序范围
- 围绕内存、文件 I/O、管道与 SMP 路径进行针对性性能与稳定性优化

自强、弘毅、求是、拓新





1.4 架构重构：构建画像与编译解耦





1.5 DTB 驱动的运行时硬件发现

① 固件输入

启动入口只接收固件传入的 DTB 物理地址，并转换为统一 BootInfo。



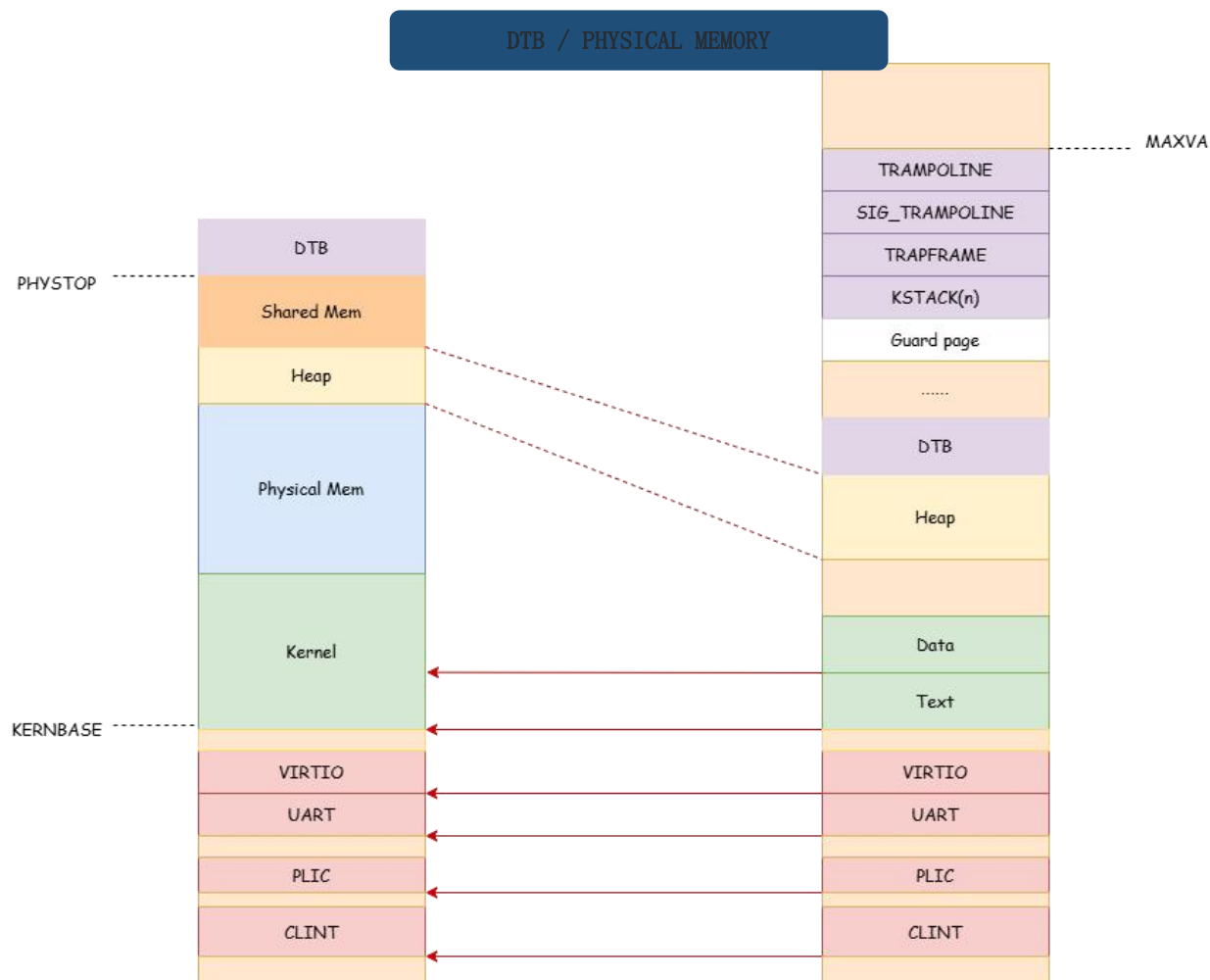
② 统一解析

DtbManager 解析 memory / reserved-memory、CPU hart、timebase、PLIC context、initrd 与网卡 MAC 等事实。



③ 子系统消费

PMM 排除 DTB 与保留区后建立可分配 RAM；SMP 只启动 DTB 声明的 CPU，设备层不再重复硬编码拓扑。



同一个硬件事实只保留一个权威来源：CPU/内存来自 DTB，板上固定资源来自平台画像，运行状态归设备驱动。

02

工作1: Linux语义实现

Linux Semantics Implementation



2.1 LTP 验证规模：297 → 786



297

2025-08 启用测例
RISC-V / LoongArch

786

2026-08 启用测例
RV/LA × musl/glibc

+489

一年新增
约 2.6× 覆盖规模

0

去年启用项移除
保持零回退

验证规模变化

2025



297

2026



786

验证内容深化

2025：仅完成basic的syscall内容对应的ltp验证

2026：扩展更多操作系统通用语义

从“能进入 syscall”推进到“在多架构、多 libc 条件下保持一致的 Linux 行为语义”。



2.2 语义增量：从 syscall 覆盖到行为语义



文件 / 挂载

95 bind/rbind/move · **48** fcntl · **87** 常规文件

MS_REC / 传播属性 · 记录锁 / pipe
size · rename / fallocate / xattr

进程 / IPC / 权限

39 进程线程 · **14** 信号 · **22** SysV SHM

clone3 / execveat / waitid · rt
signal · robust futex · capability

事件通知 / 时间

18 epoll · **12** fanotify · **7** inotify · **7** eventfd

EPOLLET / pwait · 文件事件 mask · signalfd /
timerfd · clock_*

内存 / 零拷贝

21 mmap/内存 · **7** splice

MAP_FIXED / SHARED / PRIVATE · msync /
mlock · memfd · SPLICE_F_*

网络 / 设备 / libc

19 socket · **7** ioctl · **11** libc/系统信息

accept4 / sendmmsg / socket
option · SIOCGIF* · musl / glibc 适配

共同目标：提升Linux ABI 的“入口兼容”，并由 LTP 复核，兼容各类linux用户软件程序。

03

工作2：网络链路

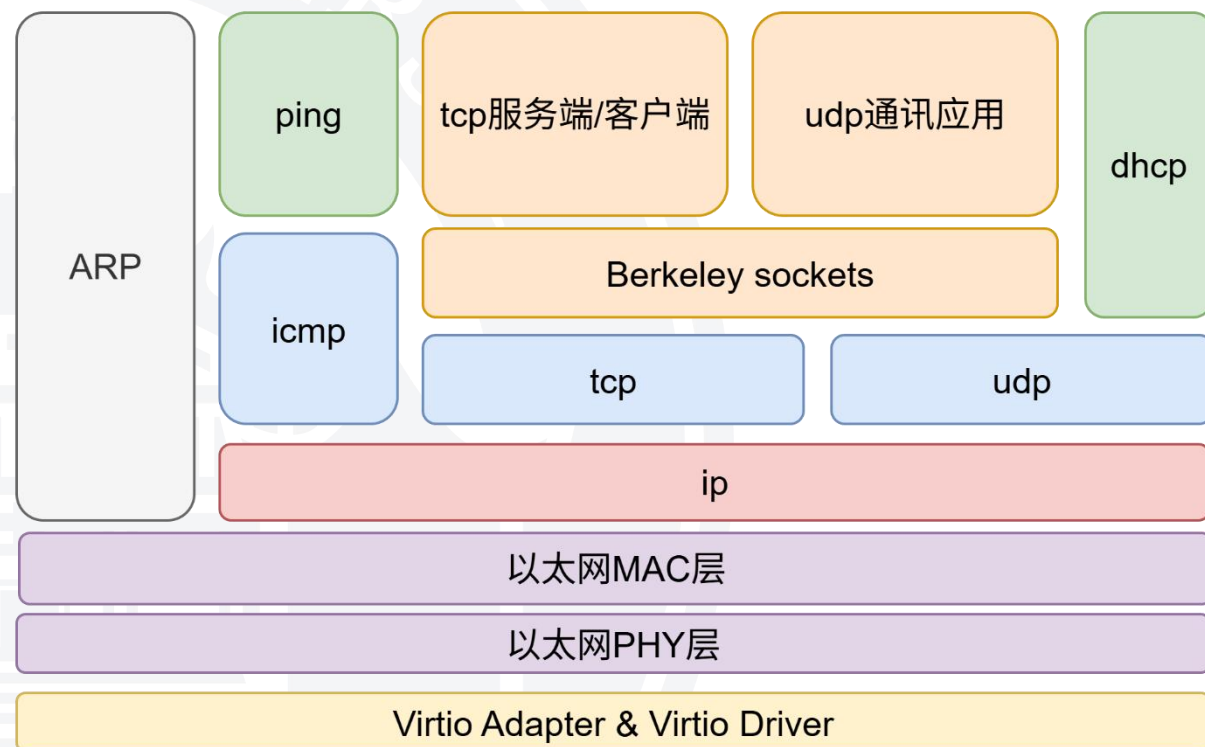
Network Link

3.1 网络模块——分层架构

网络系统基于 Open-NPStack 协议栈实现，旨在构建高效灵活的网络通信能力。本系统支持 AF_INET、AF_UNIX 等多种地址族的套接字操作，完整实现了 TCP 与 UDP 传输协议。网络系统通过统一的抽象接口管理所有网络设备与套接字资源。

整个网络系统主要包含以下几个核心组件：

1. VFS 集成层
2. BSD Socket 接口
3. 网络协议协议栈
4. 网络适配层
5. VirtIO 网络驱动层



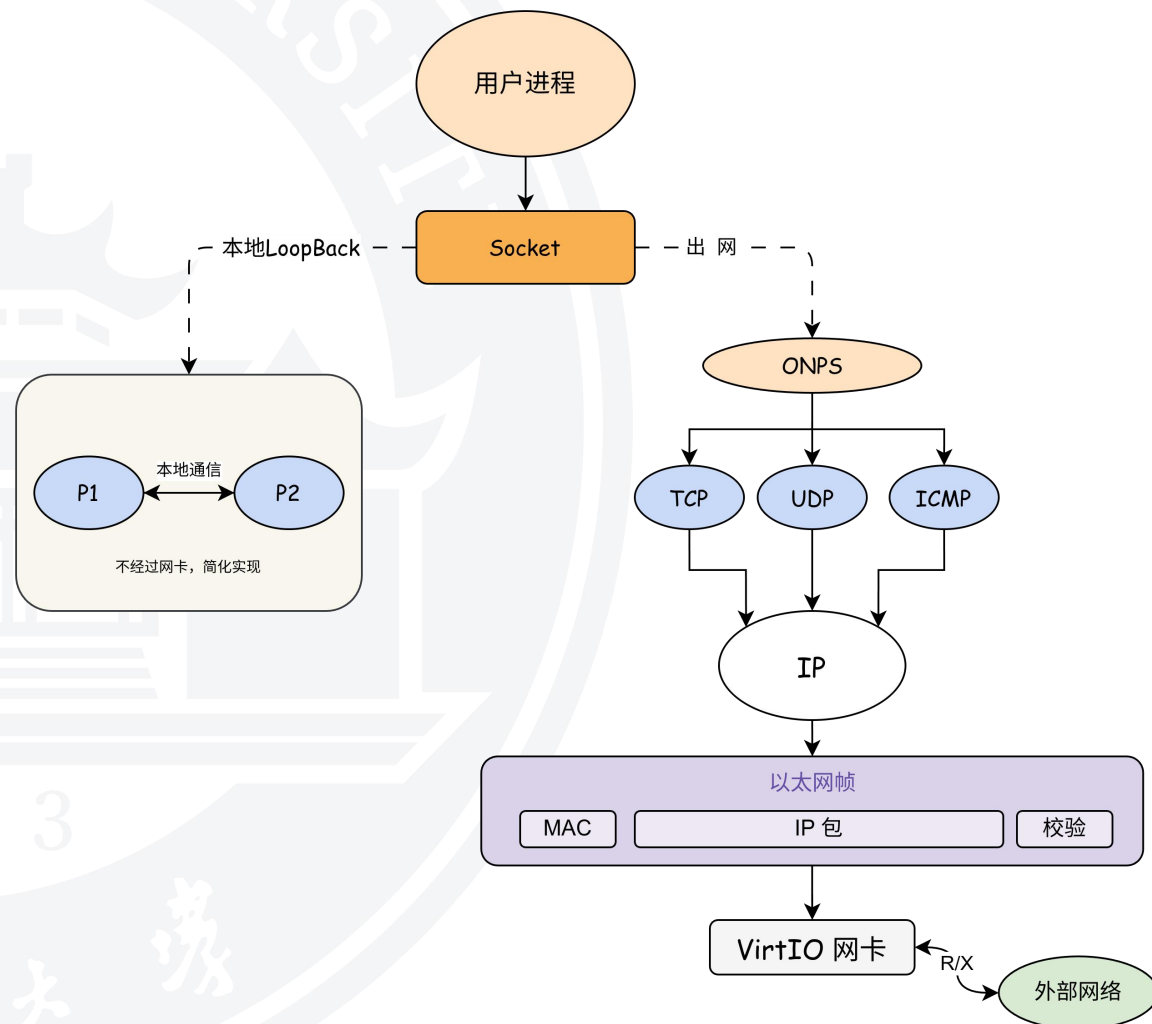
网络系统采用分层架构设计，从底层硬件驱动到上层应用接口，形成了完整的网络协议栈。

3.2 网络模块——网络协议栈与VirtIO-net

网络模块以 Linux 兼容 Socket 为统一入口，向下分为两条数据路径：

1. 本机通信走 socket_file loopback 快路径，直接通过端口表、peer 连接和缓冲队列传递 TCP/UDP payload；
2. 外部 IPv4 流量交由 ONPS 协议栈处理 TCP/UDP/ICMP/IP，再通过 VirtIO-Net 收发完整以太网帧。

该设计兼顾可验证的本机数据面与后续外网扩展能力。



04 工作3：多核代码

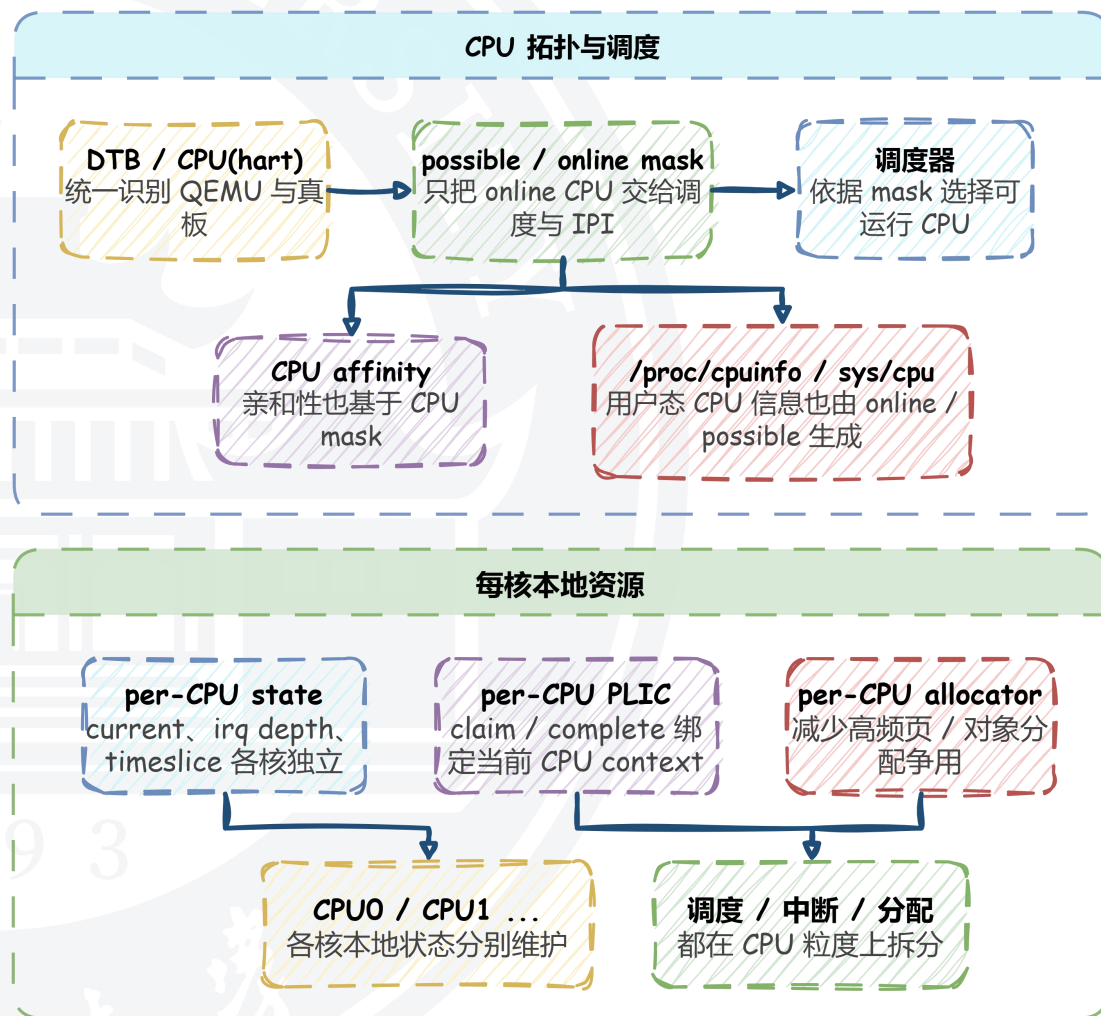
Support for multi-core functions

4.1 多核CPU初始化

多核初始化解决的是多核计算资源的虚拟化。

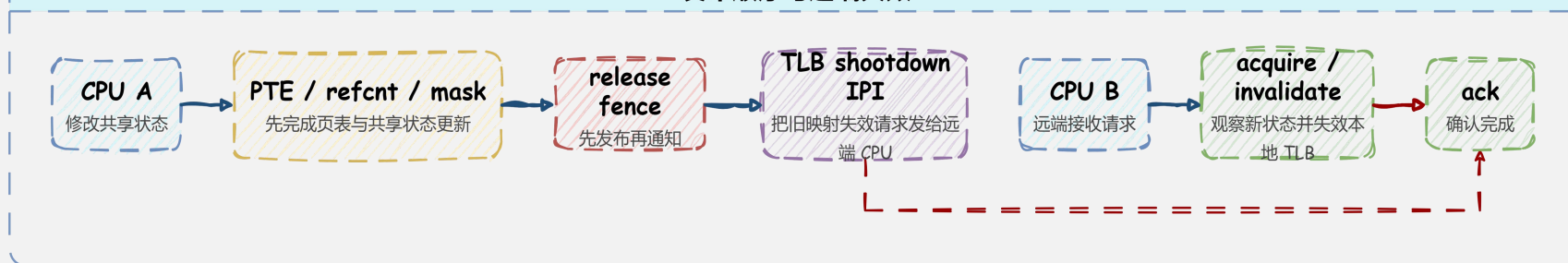
1. CPU 完成自己的中断、时钟和调度初始化后，才能进入 online 集合。调度器不能把任务分给没有成功上线的 CPU。
2. 系统中只有一份在线 CPU 事实，内核调度和用户态查询不会互相矛盾。
3. 通过记录器将高频页和对象分配尽量留在本核降低共享分配器热点。

这种设计统一 CPU 视图和调度状态，是多核的计算资源通过虚拟化的方式进行整合。

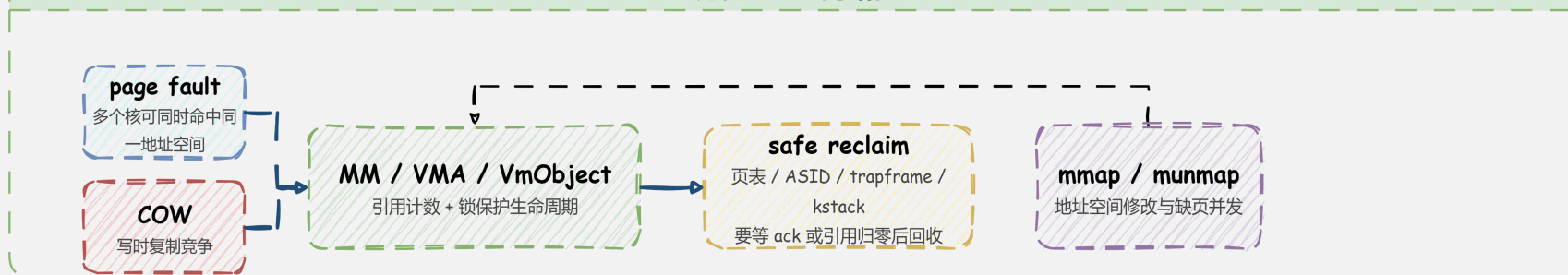


4.2 多核正确性保障 – 映射一致

发布顺序与远端失效



并发地址空间路径

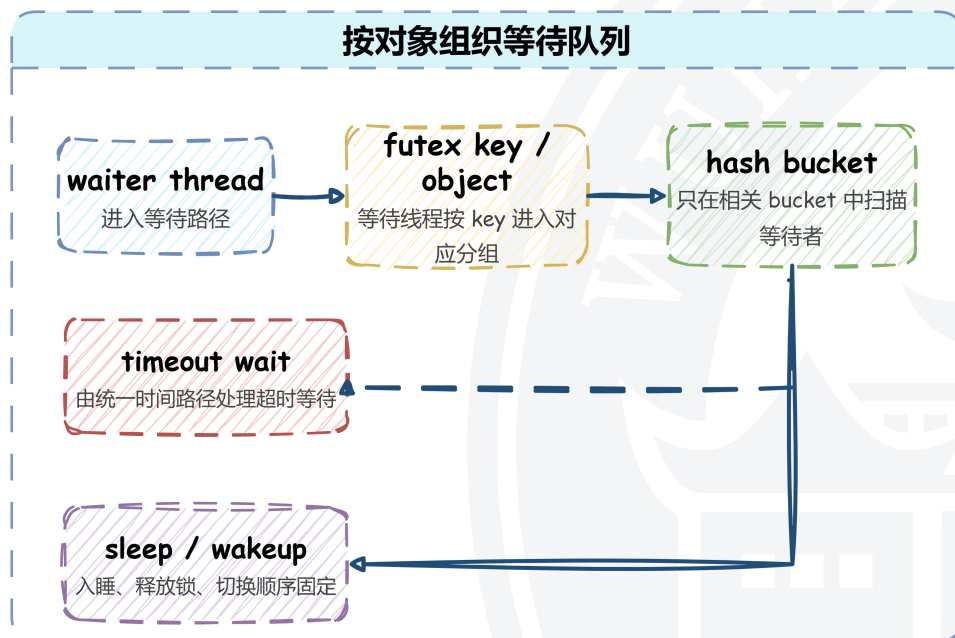


多核页表共享不等于缓存共享，因此一个 CPU 修改映射后，其他 CPU 仍可能使用旧结果，导致访问错误页面。

只有相关 CPU 完成映射失效后，旧页面和地址空间资源才能安全回收。

通过建立映射修改与资源回收明确的先后关系，避免其他 CPU 继续访问已经失效的页面。

4.3 多核正确性保障 – 可靠唤醒



分桶等待队列

futex key / 等待对象进入对应 bucket
唤醒时只扫描相关等待者

sleep / wakeup 顺序

入睡、解锁、切换顺序固定
唤醒时同时更新线程状态与等待队列

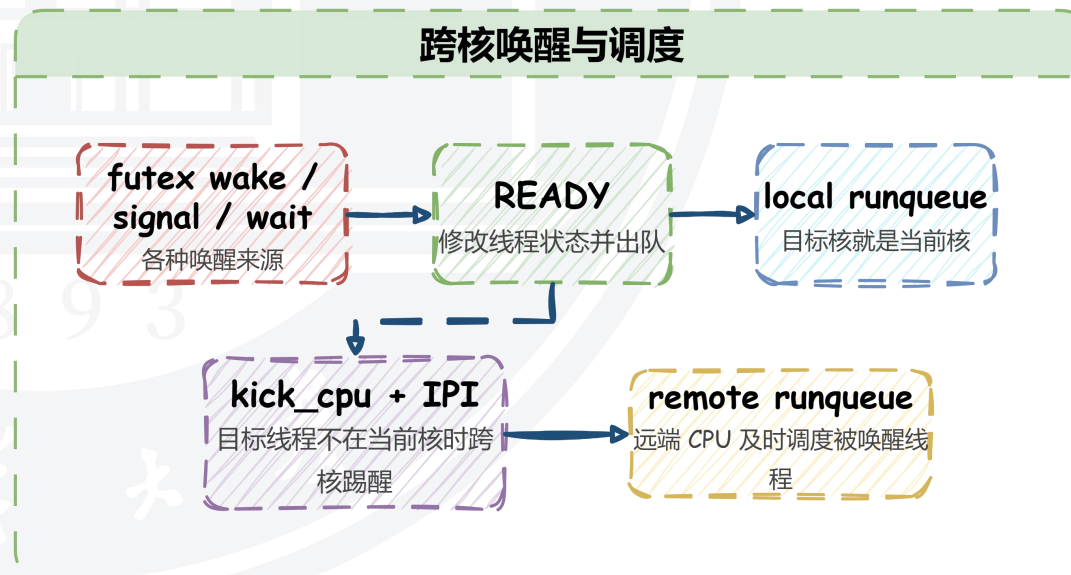
跨核唤醒

目标线程不在当前核时调用 kick_cpu
通过 IPI 让远端 CPU 及时调度

超时等待优化

由统一时间路径检查到期等待者
不再每个 tick 广播唤醒所有线程

跨核唤醒与调度

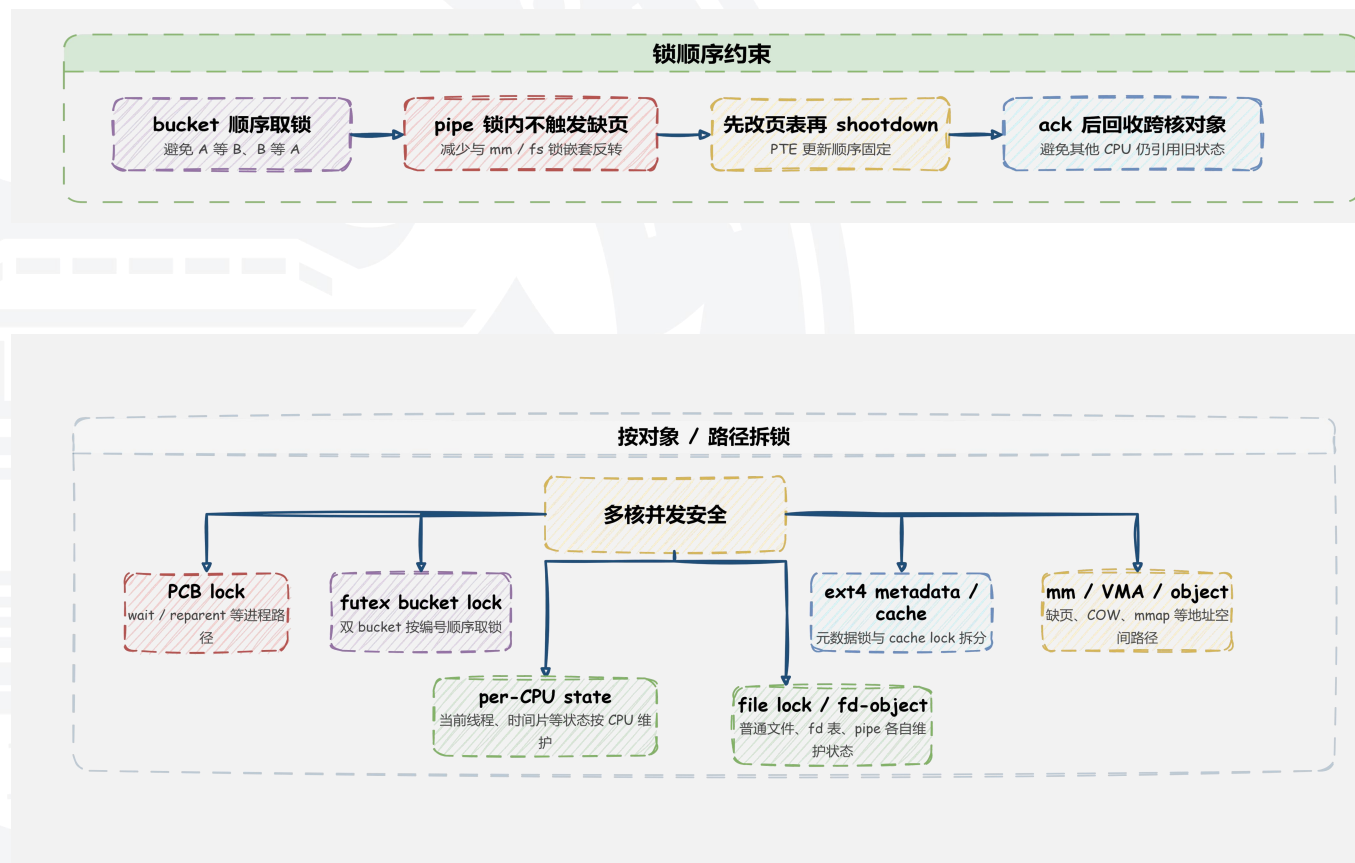


4.4 多核正确性保障 – 锁序固定

多核场景下需要特别考虑并发锁问题。

传统一把大锁的形式会让无关操作互相阻塞；但锁拆分以后，如果多个 CPU 以不同顺序获取多把锁，又可能形成循环等待。因此进行特殊设计：

1. 锁归属于具体对象，对象分别保护自己的状态
2. 跨对象操作固定顺序，统一顺序加锁和释放



05

工作4：文件系统优化

File System Optimization

5.1 VMA 记录重构：索引和后端解耦

1 VmArea 描述符

VmArea 只描述区间、权限、映射类型、页偏移和增长策略，是地址空间的“语义记录”。

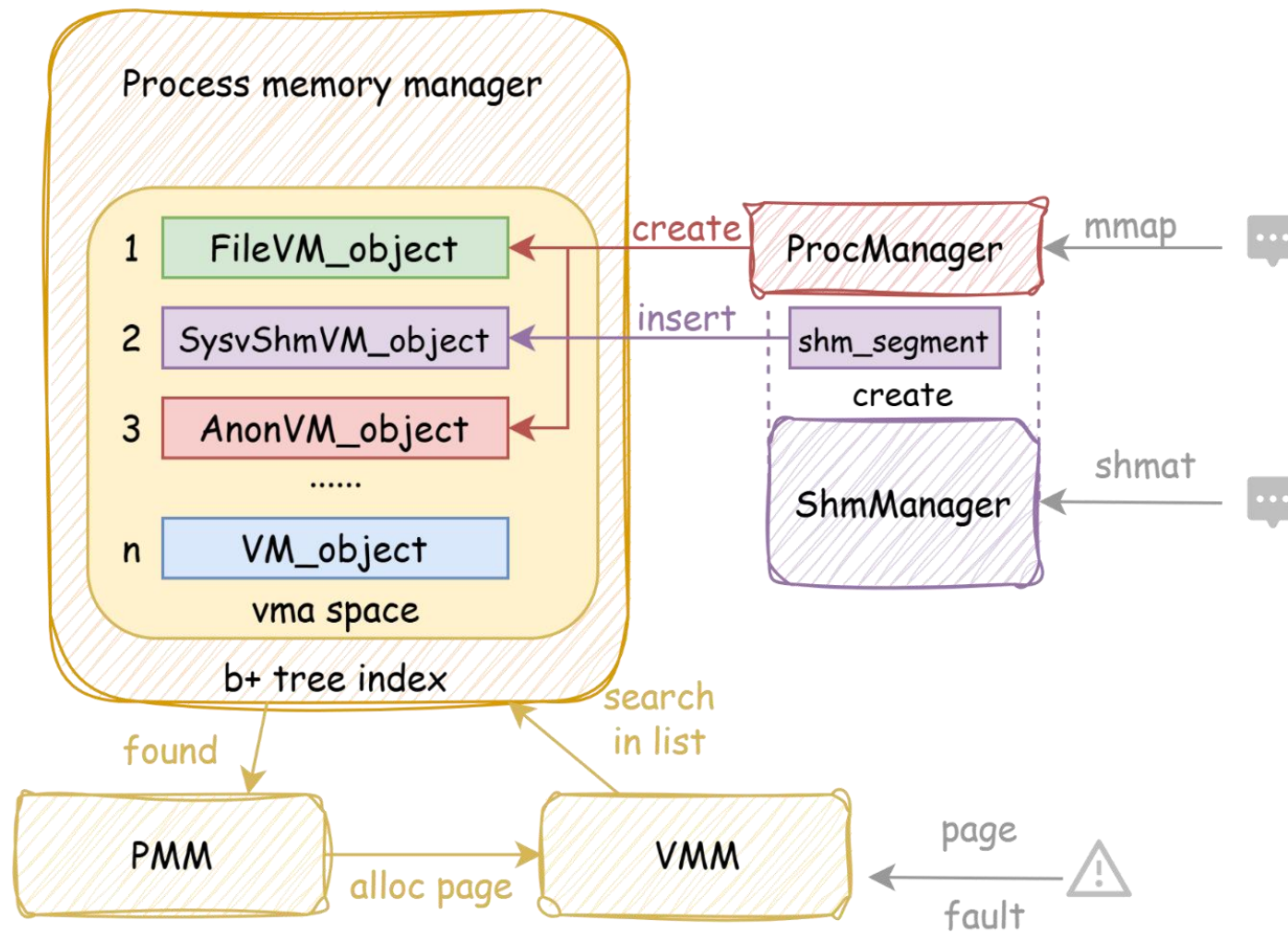
2 VMASpace + Maple Tree

VMASpace 统一负责创建、拆分、合并与 gap 查找；
Maple Tree 维护区间索引，list 保留稳定对象生命周期。

3 VmObject 后端

Anon / File / SysV SHM 三类 VmObject 各自实现 prepare_page，将匿名页、文件页缓存和共享内存的缺页语义放到对应后端。

mmap / mprotect / munmap / page fault 共享同一套 VMA 语义源，减少跨路径状态不一致。

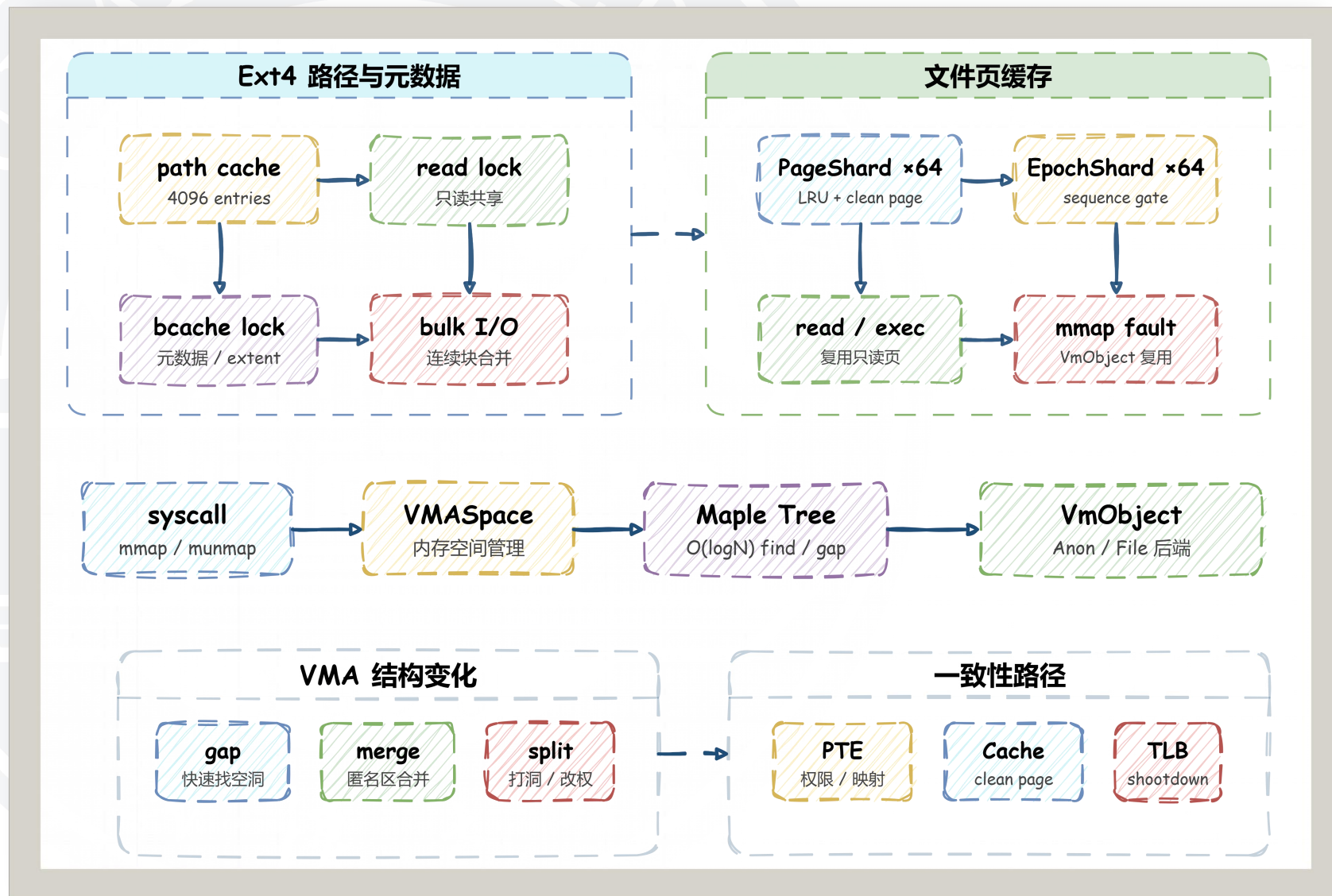


5.2 read、mmap 与 exec 复用同一干净文件页

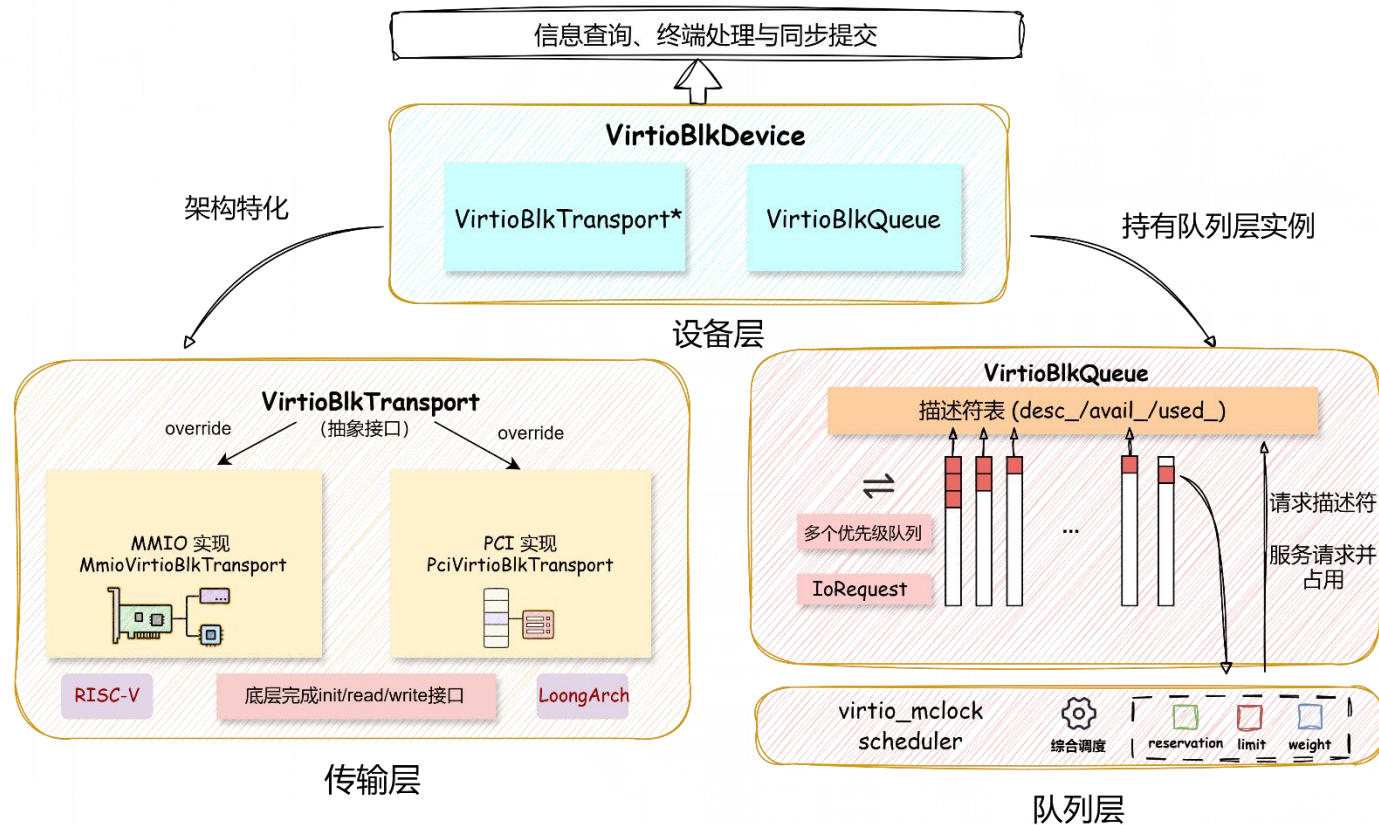
read、mmap 和 exec 可能沿不同路径重复读取同一文件内容，产生重复 I/O。

当前设计使这些文件读取链路共享全局干净文件页，并在私有映射需要写入时，通过 COW 创建当前进程自己的副本。

该设计让不同文件访问路径共享同一份可信内容，在减少磁盘读取的同时保持数据一致性。



5.3 I/O 路径减少设备空闲与小请求开销



Ext4 经常产生连续的 4 KiB 块请求。单块提交，通知设备和等待完成的固定成本会很高。

因此将连续的小块访问合并为较大的 I/O 请求，减少频繁提交和等待带来的固定开销。

并允许多个请求同时处于设备队列中，在前一个请求完成前继续派发后续请求，减少设备空闲。

07

开发心得与总结

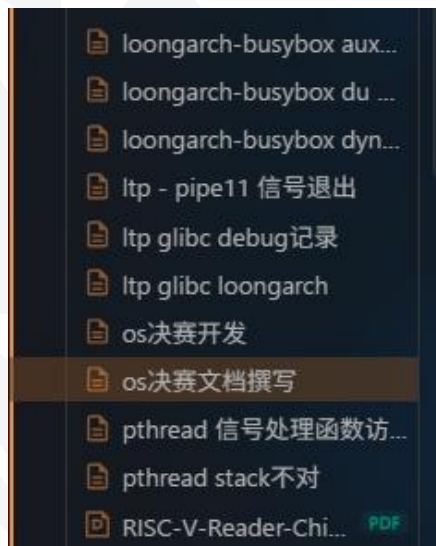
Development Insights & Summary



3.3 经验总结——问题记录



仔细记录每一次调试过程，形成记录文档，遇到重复问题时可查阅



```
/**
 * @brief 释放堆内存
 *
 * 释放进程堆内存占用的所有页面，将堆大小重置为0。
 *
 * 注意：此函数内部调用cleanup_heap_to_size(0)，仅在free_all_memory()中内部调用
 */
void free_heap_memory();

/**
 * @brief 清理堆内存到指定大小
 * @param new_size 新的堆大小，如果为0则完全释放堆内存
 * @return true 成功，false 失败
 */
bool cleanup_heap_to_size(uint64 new_size);

/*****
 * VMA管理接口
 *
 * 说明：VMA管理现在统一通过free_all_memory()进行，不建议单独调用
 *****/
```

详细为函数书写注释。使用doxygen风格注释，方便使用时参考和后续查阅。未测试\未完成的代码明确标注TODO，便于后续发现问题。

```
#elif defined(LOONGARCH)
/// TODO:未测试正确性
void *mem;
uint64 a;
// printfCyan("[vmalloc] oldsz
if (newsz < oldsz)
```


3.3 经验总结——调试日志

```
// 使用匿名命名空间限制 disable_printf 变量的作用域
namespace {
    bool disable_printf_flag = false; // 默认关闭 printf
}
```

```
void Printer::enable_printf()
{
    disable_printf_flag = false;
}
```

```
void Printer::disable_printf()
{
    disable_printf_flag = true;
}
```

```
void Printer::print(const char *fmt, ... )
{
    // 如果禁用了 printf 输出，则直接返回
    if (disable_printf_flag) {
        // 仍需要处理可变参数以避免潜在问题
        va_list ap1;
        va_start(ap1, fmt);
        va_end(ap1);
        return;
    }
}
```

可在特定测例中使用打印使能，只观察单个测例的调试输出

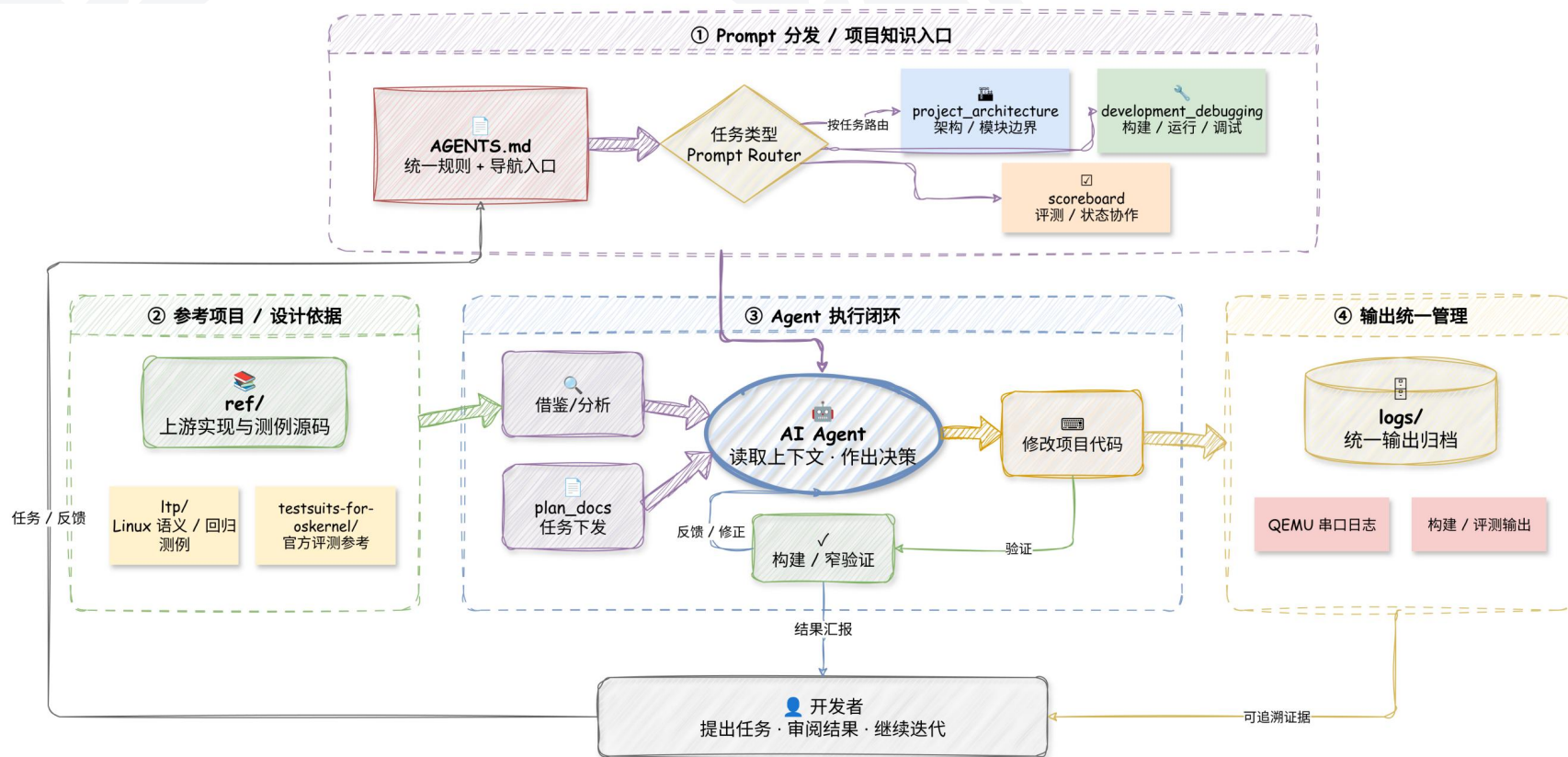
```
#define printfRed(format, ...) k_printer.print("
#define printfGreen(format, ...) k_printer.print(
#define printfBlue(format, ...) k_printer.print("
#define printfCyan(format, ...) k_printer.print('
#define printfYellow(format, ...) k_printer.print
#define printfWhite(format, ...) k_printer.print
#define printfMagenta(format, ...) k_printer.pr
// 颜色太少了，我给你加几个
#define printfBlack(format, ...) k_printer.print(
#define printfOrange(format, ...) k_printer.prin
#define printfPurple(format, ...) k_printer.print
#define printfPink(format, ...) k_printer.print("
#define printfBrown(format, ...) k_printer.print
#define printfGray(format, ...) k_printer.print('
#define printfLightRed(format, ...) k_printer.pr
#define printfLightGreen(format, ...) k_printer.
#define printfLightBlue(format, ...) k_printer.p
#define printfLightCyan(format, ...) k_printer.p
#define printfLightYellow(format, ...) k_printer
#define printfLightMagenta(format, ...) k_print
// Background colors
#define printfBgRed(format, ...) k_printer.print
#define printfBgGreen(format, ...) k_printer.pr
#define printfBgBlue(format, ...) k_printer.print
#define printfBgYellow(format, ...) k_printer.pr
#define printfBgCyan(format, ...) k_printer.prin
#define printfBgMagenta(format, ...) k_printer.
```

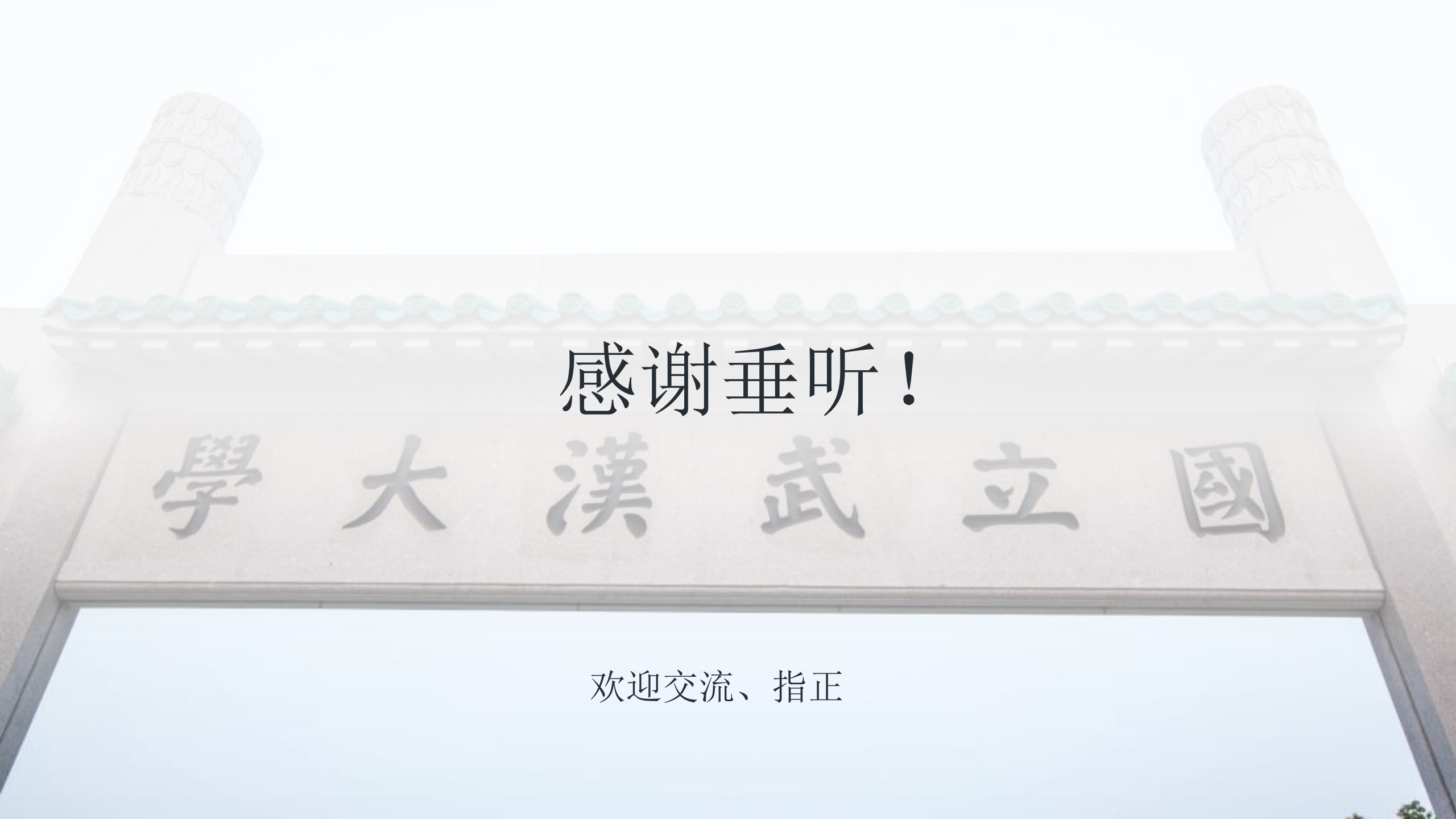
打印输出有多种色彩便于观察。警告使用黄色、完成使用绿色、错误信息使用红色……

使用多层printf开关，可灵活控制是否打开调试输出。Panic的输出不会被隐藏，运行时问题会及时暴露

3.3 经验总结——AI开发 workflow

我们将 **AI Agent** 纳入规范化开发流程：通过统一规则和任务路由建立项目上下文，结合源码与参考测例完成问题定位、代码修改和窄范围验证，并统一归档运行日志与评测结果。开发者始终负责提出任务、审查结果和决定后续迭代，使 **AI** 参与过程可追踪、可复核，而不替代工程决策。





感谢垂听！

國立武漢大學

欢迎交流、指正